Kuna Labs

Security Assessment

September 3rd, 2025 — Prepared by OtterSec

Michał Bochnak embe221ed@osec.io

Sangsoo Kang sangsoo@osec.io

alpha toure shxdow@osec.io

Table of Contents

Appendices

01 — Executive Summary

Overview

Kuna labs engaged OtterSec to assess the `leverage` and `access-management` programs. This assessment was conducted between May 29th and June 20th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 9 findings throughout this audit engagement.

In particular, we identified a vulnerability the position core does not verify that the correct Supply Pool object is utilized during liquidation, allowing a liquidator to exploit mismatched share types and receive rewards without repaying debt (OS-KUL-ADV-00). Furthermore, we highlighted an instance of precision loss due to division before multiplication when computing the amount of token X locked in the LP position for a given price and liquidity (OS-KUL-ADV-01).

We also made recommendations regarding modifications to the codebase for improved functionality and reduced redundancy (OS-KUL-SUG-05) and suggested the need to ensure adherence to coding best practices (OS-KUL-SUG-06). Moreover, we advised adding share type assertions to prevent incorrect dept repayments (OS-KUL-SUG-01), and incorporating additional safety checks within the codebase to mitigate potential security issues (OS-KUL-SUG-04).

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/kunalabs-io/sui-smart-contracts. This audit was performed against 8eb311b.

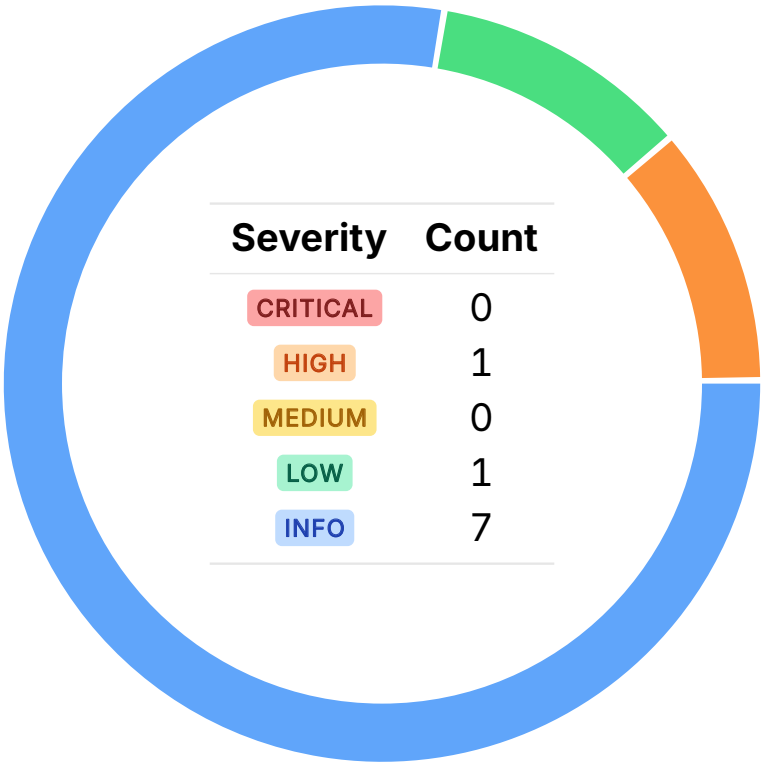
Brief descriptions of the programs are as follows:

Name	Description
access-management	A flexible and extensible `access control` framework designed for `Sui` packages.
leverage	Enables users to create and manage leveraged concentrated liquidity positions in CLMM protocols such as Cetus and Bluefin.

03 — Findings

Overall, we reported 9 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	1
MEDIUM	0
LOW	1
INFO	7

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-KUL-ADV-00	HIGH	RESOLVED	`position_core` does not verify that the correct `SupplyPool` object is utilized during liquidation, allowing a liquidator to exploit mismatched share types and receive rewards without repaying debt.
OS-KUL-ADV-01	LOW	RESOLVED	`x_by_liquidity_x64` performs division before multiplication, which may result in precision loss and return `x_x64 = 0` even for large liquidity.

Unfair Rewards via Incorrect Supply Pool Instance HIGH OS-KUL-ADV-00

Description

Functions in `position_core` lack validation to ensure the correct `SupplyPool<X, SX>` is utilized during liquidation. This is especially relevant as it is technically possible for multiple `SupplyPool<X, SX>` instances to exist for a given `X` token and with different `SX` types. Thus, if a user borrowed from `SupplyPool<X, SX0>` to create a position, a malicious liquidator may exploit this by passing a different `SupplyPool` instance than the one utilized when the position was created.

>_ *kai/leverage/core/sources/clmm/position_core.move* RUST

```
public(package) macro fun liquidate_col_y<$X, $Y, $SX, $LP>(
    $position: &mut Position<$X, $Y, $LP>,
    $config: &PositionConfig,
    $price_info: &PythPriceInfo,
    $debt_info: &DebtInfo,
    $repayment: &mut Balance<$X>,
    $supply_pool: &mut SupplyPool<$X, $SX>,
    $clock: &Clock,
): Balance<$Y> {
    [...]
    let (repayment_amt_x, reward_amt_y) = model.calc_liquidate_col_y(
        p_x128,
        repayment.value(),
        config.liq_margin_bps(),
        config.liq_bonus_bps(),
        config.base_liq_factor_bps(),
    );
    if (repayment_amt_x == 0) {
        return balance::zero()
    };
    let mut r = repayment.split(repayment_amt_x);

    let mut debt_shares = position.debt_bag_mut().take_all();
    let (_, x_repaid) = supply_pool.repay_max_possible(&mut debt_shares, &mut r, $clock);
    [...]
}
```

Consequently, the liquidator may pass `SupplyPool<X, SX1>` to `liquidate_col_y`. Since the position's debt shares are of type `SX0`, attempting to repay with `SX1` results in no repayment, but the function still proceeds to grant rewards. This results in the liquidator receiving collateral without reducing any debt.

Remediation

Validate that the passed-in `SupplyPool<X, SX>` instance is the one associated with the position.

Patch

Resolved in 6116084 and 02b13e0.

Precision Loss in Calculation of Token X Amount LOW OS-KUL-ADV-01

Description

When computing the amount of token `X` locked in the LP position for a given price and liquidity, `position_model::x_by_liquidity_x64` performs the calculation by dividing before multiplying (`x_x64 = delta_l * (num / denom)`). This may result in a loss of precision when `num < denom`, resulting in `(num / denom)` to round down to zero. As a result, `x_x64` becomes zero even if `delta_l` is large, incorrectly implying no `X` is in the position. This affects margin calculations, deleveraging, and liquidation logic.

>_ *kai/leverage/core/sources/clmm/position_model.move* RUST

```rust
/// Calculate the amount of X in the LP position for a given price and liquidity.
/// Aborts if there's not enough liquidity in the position.
public fun x_by_liquidity_x64(self: &PositionModel, sqrt_p_x64: u128, delta_l: u128): u128 {
    [...]
    // L * (sqrt(pb) - sqrt(p)) / (sqrt(p) * sqrt(pb))
    let num = ((self.sqrt_pb_x64 - sqrt_p_x64) as u256) << 128;
    let denom = (sqrt_p_x64 as u256) * (self.sqrt_pb_x64 as u256);

    let x_x64 = (delta_l as u256) * (num / denom);
    [...]
}
```

Remediation

Reorder the expression to perform multiplication first (`(delta_l * num) / denom`), preserving numerical accuracy.

Patch

Resolved in ad08f8c.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-KUL-SUG-00	`increase_value_and_issue_x64` allows minting unbacked equity shares if the registry is reset to zero, enabling a malicious user to drain real assets from a newly created `SupplyPool`.
OS-KUL-SUG-01	Certain functions lack share type checks, risking incorrect debt repayments.
OS-KUL-SUG-02	`rebalance_collect_fee` fails to verify that the `pool_object` matches the LP in the position.
OS-KUL-SUG-03	`repay_lossy` may zero out `liability_value_x64` while shares remain, and `increase_liability_and_issue_x64` may overwrite those shares, resulting in the loss of existing debt share value.
OS-KUL-SUG-04	There are several instances where proper validation checks may be incorporated within the codebase to make it more robust and secure.
OS-KUL-SUG-05	Recommendation for updating the codebase to improve functionality and reducing redundancy.
OS-KUL-SUG-06	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.

Unbacked Equity Share Minting OS-KUL-SUG-00

Description

It is possible for a malicious user to initialize a new `EquityRegistry` and mint equity shares via `increase_value_and_issue`. The attacker may reduce `underlying_value_x64` to zero via `decrease_value_x64`, resetting the registry. The user then calls `increase_value_and_issue_x64` with `value_x64` passed as zero, and since `underlying_value_x64 == 0`, the function sets both `underlying_value_x64` and `supply_x64` to zero.

>_ *kai/leverage/core/sources/primitives/equity.move* RUST

```
public fun increase_value_and_issue_x64<T>(
    registry: &mut EquityRegistry<T>,
    value_x64: u128,
): EquityShareBalance<T> {
    if (registry.underlying_value_x64 == 0) {
        registry.underlying_value_x64 = value_x64;
        registry.supply_x64 = value_x64;
        return EquityShareBalance { value_x64 }
    };
    [...]
}
```

With this forged registry and treasury, the user may utilize that `EquityTreasury<T>` to create a new `SupplyPool` object. Consequently, the malicious user will hold an arbitrary number of shares, which they may later burn and completely drain the `SupplyPool`.

Remediation

Add a guard in `increase_value_and_issue_x64` to ensure `value_x64 > 0` if `registry.underlying_value_x64 == 0`.

Patch

Resolved in 76b2dc7.

Missing Share Type Validation OS-KUL-SUG-01

Description

Currently in `position_core`, `deleverage_ticket_repay_x` and `deleverage_ticket_repay_y` lack critical type checks to ensure that the `SupplyPool`'s share type matches the debt share type recorded in the position's `DebtBag`. Without this assertion, there is a risk of repaying debt with the wrong share token, resulting in corrupted accounting. In contrast, the `liquidate_col_x` and `liquidate_col_y` already include these safety assertions to enforce share type correctness. To maintain protocol integrity, the same type checks should be added to the deleveraging functions and extended to all functions interacting with `SupplyPool` objects.

Remediation

Ensure to add share type assertions to `deleverage_ticket_repay_x` and `deleverage_ticket_repay_y`, and also to all functions that interact with `SupplyPool` objects.

Patch

Resolved in d5e3404.

Absence of Pool-Position Validation OS-KUL-SUG-02

Description

`position_core::rebalance_collect_fee` assumes the provided `pool_object` matches the LP position stored in the position, but it does not validate this relationship. If an incorrect `pool_object` is passed, and the `collect_fee` closure does not enforce the match, fees may be collected from unrelated or empty pools, resulting in incorrect fee accounting.

Remediation

Ensure `rebalance_collect_fee` asserts that the LP position is linked to the given pool.

Patch

Resolved in 3d7cb98.

Risk of Debt Share Dilution

OS-KUL-SUG-03

Description

`debt::repay_lossy` may reduce `liability_value_x64` to zero while leaving `supply_x64` (debt shares) non-zero due to rounding behavior. However, `increase_liability_and_issue_x64` blindly resets the share supply when it sees `liability_value_x64 == 0`, assuming the registry is empty. This results in any remaining debt shares in `DebtRegistry<T>` to be overwritten and effectively erased. As a result, users holding valid shares may lose their value.

>_ *kai/leverage/core/sources/primitives/debt.move* RUST

```rust
public fun increase_liability_and_issue_x64<T>(
    registry: &mut DebtRegistry<T>,
    value_x64: u128,
): DebtShareBalance<T> {
    if (registry.liability_value_x64 == 0) {
        registry.liability_value_x64 = value_x64;
        registry.supply_x64 = value_x64;
        return DebtShareBalance { value_x64 }
    };
    [...]
}
```

Remediation

Ensure that `supply_x64 == 0` before resetting it.

Patch

The Kuna Labs team acknowledged this issue.

Missing Validation Logic OS-KUL-SUG-04

Description

1. Ensure that the `SupplyPool`'s asset type `T` matches either `X` or `Y`, and that its share type `ST` aligns with the expected debt share type to prevent misconfigurations.
2. `piecewise::create` should validate if sections are sorted by `end` to prevent aborts in `value_at`.
3. In the current implementation, it is possible for a single pool to have multiple configs. As a result, core functions in `position_core` may behave inconsistently across position lifecycle stages. Restrict a pool to a single config.

Remediation

Update the codebase with the above checks.

Patch

1. Issue #1 resolved in d5e3404.
2. Issue #2 resolved in be2918c.

Code Refactoring OS-KUL-SUG-05

Description

1. `piecewise::value_at` already includes an early return when `x == pw.start`, but lacks a similar check for `x == last_section.end`. Adding an early exit for this case will avoid unnecessary looping and interpolation.
2. `utils::timestamp_sec` returns time in seconds by truncating millisecond precision, which may reduce accuracy. Downstream logic will benefit from retaining full millisecond precision.

>_ kai/leverage/core/sources/util.move RUST

```rust
/// Get current clock timestamp in seconds.
public fun timestamp_sec(clock: &Clock): u64 {
    clock::timestamp_ms(clock) / 1000
}
```

Remediation

Incorporate the above refactors.

Code Maturity OS-KUL-SUG-06

Description

1. Add overflow checks in `divide_and_round_up_u128` and `divide_and_round_up_u256` in `util`, as they may overflow when computing `a + b - 1` if `a+b` is close to the maximum value of the type (`u128::max` or `u256::max` respectively).

   >_ *kai/leverage/core/sources/util.move* RUST

   ```
   public fun divide_and_round_up_u128(a: u128, b: u128): u128 {
       (a + b - 1) / b
   }

   public fun divide_and_round_up_u256(a: u256, b: u256): u256 {
       (a + b - 1) / b
   }
   ```

2. Several setter functions lack input validation, which may result in unsafe or illogical configuration values. Add sanity checks to ensure values remain within acceptable bounds.

Remediation

Implement the above-mentioned suggestions.

Patch

1. Issue #1 resolved in 0a779eb.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.